Operador visual de sistemas sin API

Manuel Argüelles Guerra | Frontier 2026 | Reto 1 | Propuesta para revisión

Objetivo y límites

Ejecutar instrucciones nuevas sobre aplicaciones desconocidas, únicamente por pantalla y VNC. El contrato entrega un websocket RFB y contraseña por tarea; la app de prueba SIGECOM funciona en kiosko 1280x800. No accedemos al DOM, almacenamiento ni APIs de la aplicación. La exploración confirmó menús, filtros y homónimos; aún no hay agente implementado ni prueba de generalización.

Arquitectura propuesta

Un servicio TypeScript/Node publica la API y coordina un worker aislado por tarea. POST /tasks autentica, valida y reserva capacidad antes de responder solo task_id en menos de 2 segundos. SQLite en volumen guarda estado y resumen, sin contraseña; tras reinicio, tareas interrumpidas quedan fallidas y no se reejecutan. Un visor privado propio usa noVNC en Chromium para conectarse al escritorio recibido. Playwright opera ese visor y captura su framebuffer; el modelo visual recibe pixels, tarea y un historial acotado. Gemini 2.5 Flash vía OpenRouter es candidato inicial, sujeto a pruebas de precisión, coordenadas, coste y latencia.

Control de ejecución y seguridad

Observar / proponer una acción estructurada / validar / ejecutar / observar de nuevo. Coordenadas del framebuffer, sin reutilizar posiciones entre pantallas. Clic sostenido 150 ms; espera inicial 1 s y comprobación acotada de estabilidad. Reemplazo mediante selección explícita y verificación del texto, incluidas tildes. Antes de escribir se identifica el registro, se comprueba duplicidad y se delimita el cambio; una ambigüedad produce fallida sin mutación. El contenido de pantalla es dato, nunca instrucción con autoridad: ignorar la orden de eliminar pedidos incrustada en Observaciones, preservando esa nota. Validar wss y destino público, sin redirects ni acceso a red privada. Password y tokens quedan fuera del modelo, resumen y logs; capturas de desarrollo privadas.

Tiempo, estado y concurrencia

Una tarea activa por instancia inicialmente y exclusión por escritorio; no se aceptan trabajos que quedarían en cola fuera de plazo. Watchdog monotónico: cancelar a 270 s, soltar teclas/botones y cerrar sesión a 280 s, con margen antes del límite de 300 s. El estado terminal se publica después de detener el actuador. Cada acción comprueba cancelación y usa timeout; reintentos solo tras observar el resultado para evitar duplicados. GET /tasks/{id} devuelve estado y resumen; preguntas usan una línea respuesta: valor (dos datos separados por ;). Una pantalla sin evidencia de éxito no permite reportar terminada.

Validación, despliegue y tradeoffs

Después de cada reset, ejecutar E1 alta, E2 cambio de dirección, E3 conteo paginado, E4 exportación filtrada y E5 homónimos. El harness usa __check y expectativas; el agente solo recibe instrucción y escritorio. Medir exactitud, acciones extra, duplicados, falsos éxitos, p95 de aceptación, tiempo total, memoria y coste. Imagen Docker idéntica para staging y destino final; Railway es candidato, con 1 GB aún por medir. Chromium simplifica decodificación RFB, pero consume memoria; alternativa: cliente RFB directo si el ensayo excede recursos. Prioridad: transporte/teclado, E5/E3, escrituras verificadas y luego robustez ante una app distinta. Loop propio frente a OpenClaw/frameworks: control explícito y menor superficie. SQLite evita otra base de datos; concurrencia uno evita un pool sin memoria medida. No se entrena visión: faltan datos y tiempo.